JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface DirectoryStream<T>

Type Parameters:

T - The type of element returned by the iterator

All Superinterfaces:

AutoCloseable, Closeable, Iterable<T>

All Known Subinterfaces:

SecureDirectoryStream<T>

```
public interface DirectoryStream<T>
extends Closeable, Iterable<T>
```

An object to iterate over the entries in a directory. A directory stream allows for the convenient use of the for-each construct to iterate over a directory.

While `DirectoryStream` extends `Iterable`, it is not a general-purpose `Iterable` as it supports only a single `Iterator`; invoking the `iterator` method to obtain a second or subsequent iterator throws `IllegalStateException`.

An important property of the directory stream's `Iterator` is that its `hasNext` method is guaranteed to read-ahead by at least one element. If `hasNext` method returns `true`, and is followed by a call to the `next` method, it is guaranteed that the `next` method will not throw an exception due to an I/O error, or because the stream has been `closed`. The `Iterator` does not support the `remove` operation.

A `DirectoryStream` is opened upon creation and is closed by invoking the `close` method. Closing a directory stream releases any resources associated with the stream. Failure to close the stream may result in a resource leak. The try-with-resources statement provides a useful construct to ensure that the stream is closed:

```
    Path dir = ...
    try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir)) {
        for (Path entry: stream) {
            ...
        }
    }
```

Once a directory stream is closed, then further access to the directory, using the `Iterator`, behaves as if the end of stream has been reached. Due to read-ahead, the `Iterator` may return one or more elements after the directory stream has been closed. Once these buffered elements have been read, then

subsequent calls to the `hasNext` method return `false`, and subsequent calls to the `next` method will throw `NoSuchElementException`.

A directory stream is not required to be *asynchronously closeable*. If a thread is blocked on the directory stream's iterator reading from the directory, and another thread invokes the `close` method, then the second thread may block until the read operation is complete.

If an I/O error is encountered when accessing the directory then it causes the `Iterator`'s `hasNext` or `next` methods to throw `DirectoryIteratorException` with the `IOException` as the cause. As stated above, the `hasNext` method is guaranteed to read-ahead by at least one element. This means that if `hasNext` method returns `true`, and is followed by a call to the `next` method, then it is guaranteed that the `next` method will not fail with a `DirectoryIteratorException`.

The elements returned by the iterator are in no specific order. Some file systems maintain special links to the directory itself and the directory's parent directory. Entries representing these links are not returned by the iterator.

The iterator is *weakly consistent*. It is thread safe but does not freeze the directory while iterating, so it may (or may not) reflect updates to the directory that occur after the `DirectoryStream` is created.

Usage Examples: Suppose we want a list of the source files in a directory. This example uses both the for-each and try-with-resources constructs.

```
    List<Path> listSourceFiles(Path dir) throws IOException {
        List<Path> result = new ArrayList<>();
        try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir, "*.{c,h
            for (Path entry: stream) {
                result.add(entry);
            }
        } catch (DirectoryIteratorException ex) {
            // I/O error encountered during the iteration, the cause is an IOExce
            throw ex.getCause();
        }
        return result;
    }
```

Since:

1.7

See Also:

`Files.newDirectoryStream(Path)`

Nested Class Summary

Nested Classes

Modifier and Type	Interface	Description
`static interface`	**`DirectoryStream.Filter<T`**	An interface that is implemented by objects that decide if a directory entry should be accepted or filtered.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`Iterator<T>`	**`iterator()`**	Returns the iterator associated with this `DirectoryStream`.

Methods declared in interface Closeable

`close`

Methods declared in interface Iterable

`forEach`, `spliterator`

Method Details

iterator

`Iterator<T> iterator()`

Returns the iterator associated with this `DirectoryStream`.

Specified by:

`iterator` in interface `Iterable<T>`

Returns:

the iterator associated with this `DirectoryStream`

Throws:

`IllegalStateException` - if this directory stream is closed or the iterator has already been returned

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.